

School Of DevOps

Python

Ivan Adj - Krstev
ENDAVA

PYTHON

Conditional statements

Iterators

Method and Functions

Inheritance

Sequence types

Sequences

In Python programming, sequences are a generic term for an ordered set which means that the order in which we input the items will be the same when we access them.

Python supports six different types of sequences.

1. strings
2. lists
3. tuples
4. byte sequences
5. byte arrays
6. range objects

Python Strings

Strings are a group of characters written inside a single or double-quotes.

Strings are immutable in nature so we can reassign a variable to a new string but we can't make any changes in the string.

```
>>> name = "DevOps - love it"
>>> type(name)
<class 'str'>
>>> city = 'Wuhan'
>>> print(city[2])
h
>>> city[2]='a'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
>>> █
```

Python String

stringliteral:	shortstring longstring
shortstring:	"" shortstringitem* "" ''' shortstringitem* '''
longstring:	""" longstringitem* """ ''' longstringitem* '''
shortstringitem:	shortstringchar escapeseq
longstringitem:	longstringchar escapeseq
shortstringchar:	<any ASCII character except "\" or newline or the quote>
longstringchar:	<any ASCII character except "\">
escapeseq:	"\" <any ASCII character>

Python List

Python lists are similar to an array but they allow us to create a heterogeneous collection of items inside a list. A list can contain numbers, strings, lists, tuples, dictionaries, objects, etc.

Lists are declared by using square brackets around comma-separated items.

Lists are mutable which makes it easier to change and we can quickly modify a list by directly accessing it.

```
>>> list1 = [1,2,3,4]
>>> list2 = ['red', 'green', 'blue']
>>> list3 = ['hello', 100, 3.14, [1,2,3] ]
>>> list = [10,20,30,40]
>>> list[1] = 100
>>> print( list)
[10, 100, 30, 40]
>>> 
```

Python Tuples

Tuples are also a sequence of Python objects. A tuple is created by separating items with a comma. They can be optionally put inside the parenthesis () but it is necessary to put parentheses in an empty tuple.

A single item tuple should use a comma in the end.

Tuples are also immutable like strings so we can only reassign the variable but we cannot change, add or remove elements from the tuple.

```
>>> tup = ()
>>> print( type(tup) )
<class 'tuple'>
>>> tup = (1,2,3,4,5)
>>> tup = ("78 Street", 3.8, 9826 )
>>> print(tup)
('78 Street', 3.8, 9826)
>>> tup = (1,2,3,4,5)
>>> tup[2] = 10
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
>>> █
```

Python Bytes Sequences

The `bytes()` function in Python is used to return an immutable bytes sequence. Since they are immutable, we cannot modify them.

If you want a mutable byte sequence, then it is better to use byte arrays. This is how we can create a byte of a given integer size.

Python Byte Arrays

Byte arrays are similar to bytes sequence. The only difference here is that byte arrays are mutable while bytes sequences are immutable. So, it also returns the bytes object the same way.

Since byte arrays are mutable, let's try changing a byte from the array.

```
>>> print( bytearray(4) )  
bytearray(b'\x00\x00\x00\x00')  
>>> print( bytearray([1, 2, 3, 4]) )  
bytearray(b'\x01\x02\x03\x04')  
>>> print( bytearray('Hola!', 'utf-8'))  
bytearray(b'Hola!')  
>>> a = bytearray([1,3,4,5])  
>>> print(a)  
bytearray(b'\x01\x03\x04\x05')  
>>> a[2] = 2  
>>> print(a)  
bytearray(b'\x01\x03\x02\x05')  
>>> █
```

Python range() objects

`range()` is a built-in function in Python that returns us a range object. The range object is nothing but a sequence of integers. It generates the integers within the specified start and stop range.

The first argument is the starting range, the second argument is the stopping range and the third argument tells how many steps to take.

```
>>> for i in range(5):  
...     print(i)  
...  
0  
1  
2  
3  
4  
>>> for i in range(4,16,2):  
...     print(i)  
...  
4  
6  
8  
10  
12  
14
```

Conditional Statements

Sequential execution, in which statements are always performed one after the next, in exactly the order specified.

Decision-making is as important in any programming language as it is in life.

Decision-making in a programming language is automated using conditional statements.

A control structure directs the order of execution of the statements in a program.

If, elif and else Statements in Python

program needs to skip over some statements, execute a series of statements repetitively, or choose between alternate sets of statements to execute.

A control structure directs the order of execution of the statements in a program - control flow

- If
- elif
- else

Control flow

Control flow syntax makes use of colons and indentation (whitespace)

This indentation system is crucial to Python and is what sets it apart from other programming languages.

This use of whitespace, an indentation allows Python code to be easily readable and very quick to prototype.

Python PEP8

Check together

Syntax check

Flake8

Indentation

if <expr>:

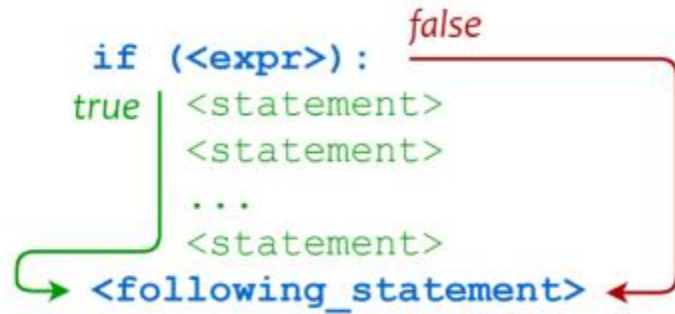
 <statement>

 <statement>

 ...

 <statement>

<following_statement>



IF

Syntax of “if” statement

if **some_condition**:

 # execute some code

else:

 # do something else

Multiple conditions

if **some_condition**:

 # execute some code

elif **some_other_condition**:

 # do something different

else:

 # do something else

One-Line if Statements

Syntax:

`value_1 if condition else value_2`

Example:

```
age = 15
```

```
print('kid' if age < 18 else 'adult')
```

Functions vs Methods

A function is a block of code to carry out a specific task, will contain its own scope and is called by name.

All functions may contain zero(no) arguments or more than one arguments.

On exit, a function can or can not return one or more values.

```
def functionName( arg1, arg2,...):  
    .....  
    # Function_body  
    ..... *
```

```
def sum(num1, num2):  
    return (num1 + num2)
```

Functions

```
def name_of_function(name):
```

```
    """
```

```
    Docstrings explains function.
```

```
    """
```

```
        print("Hello" + name)
```

Functions

Three type of functions:

Built-in functions As the name suggests, these functions come with the Python language, for example, `help()` to ask for any help, `max()`- to get maximum value, `type()`- to return the type of an object or `print()` and many more.

User-defined functions These are the functions that users create to help them, to do something that is user defined.

Anonymous Functions also called lambda functions and unlike normal function which is defined using `def` keyword are defined using `lambda` keyword.

Functions

***args and **kwargs**

Functions

`*args`

The special syntax `*args` in function definitions in python is used to pass a variable number of arguments to a function. It is used to pass a non-key worded, variable-length argument list.

- The syntax is to use the symbol `*` to take in a variable number of arguments; by convention, it is often used with the word `args`.
- What `*args` allows you to do is take in more arguments than the number of formal arguments that you previously defined. With `*args`, any number of extra arguments can be tacked on to your current formal parameters (including zero extra arguments).
- For example : we want to make a multiply function that takes any number of arguments and able to multiply them all together. It can be done using `*args`.
- Using the `*`, the variable that we associate with the `*` becomes an iterable meaning you can do things like iterate over it, run some higher-order functions such as `map` and `filter`, etc.

Functions

The special syntax `**kwargs` in function definitions in python is used to pass a keyworded, variable-length argument list. We use the name `kwargs` with the double star. The reason is because the double star allows us to pass through keyword arguments (and any number of them).

- A keyword argument is where you provide a name to the variable as you pass it into the function.
- One can think of the `kwargs` as being a dictionary that maps each keyword to the value that we pass alongside it. That is why when we iterate over the `kwargs` there doesn't seem to be any order in which they were printed out.

Methods

Methods are essentially functions that are built into objects.

The method is implicitly used for an object for which it is called.

The method is accessible to data that is contained within the class.

```
class ClassName:
    def method_name():
        ..... *
        # Method_body
        .....
```

```
class Pet(object):
    def my_method(self):
        print("I am a Cat")
cat = Pet()
cat.my_method()
```

OOP and Class

A class is a user-defined blueprint or prototype from which objects are created.

Creating a new class creates a new type of object, allowing new instances of that type to be made.

Each class instance can have attributes attached to it for maintaining its state.

A Class is like an object constructor, or a "blueprint" for creating objects.

Class creates a user-defined data structure, which holds its own data members and member functions, which can be accessed and used by creating an instance of that class. A class is like a blueprint for an object.

Class - self and `__init__()`

Class functions that begin with double underscore `__` are called special functions as they have special meaning.

Of one particular interest is the `__init__()` function. This special function gets called whenever a new object of that class is instantiated.

The `self` is used to represent the [instance](#) of the class. With this keyword, you can access the attributes and methods of the [class in python](#). It binds the attributes with the given arguments.

Object Oriented Programming

Inheritance is the capability of one class to derive or inherit the properties from another class

The benefits of inheritance are:

1. It represents real-world relationships well.
2. It provides reusability of a code. We don't have to write the same code again and again. Also, it allows us to add more features to a class without modifying it.
3. It is transitive in nature, which means that if class B inherits from another class A, then all the subclasses of B would automatically inherit from class A.